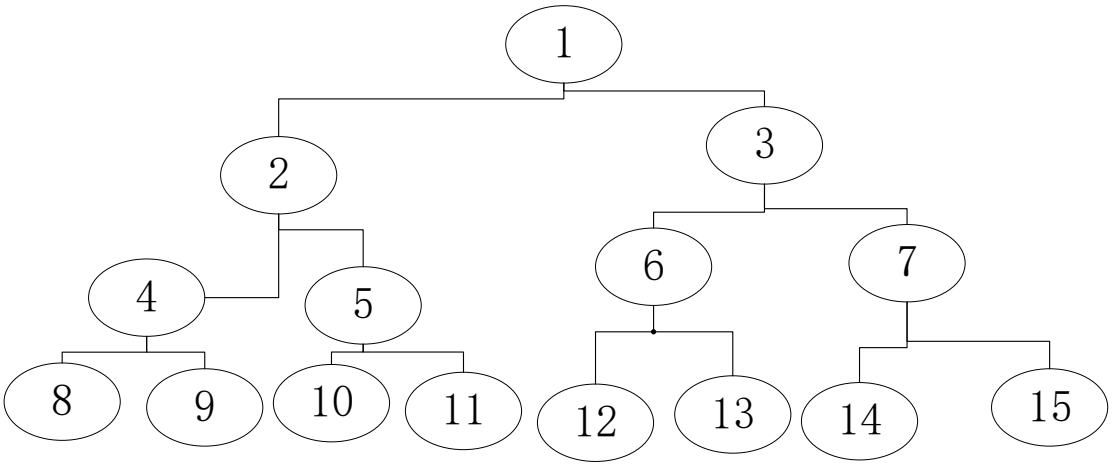
Problem1:



b.
bfs : 1- >2 ->3 - > 4 - >5 - > 6 - > 7 - >8 - > 9 -> 10 - > 11
dfs : 1- > 2 -> 4- > 8 -> 9 - >5 -> 10 -> 11
iterative: 1 (limit = 1)1->- >2 - > 3-> (limit = 2) 1- >2 - > 4 - >5 - >3 -> 6 - > 7 - >(limit = 3) 1- >2 - > 4->8 -> 9 - >5 -> 10 -> 11

problem2

a. bfs is a special case of uniform-cost search

true,
uniform-cost search expands the cheapest node first.
If every node is equal cheap, g(n) increases when depth increase, so bfs is a special case of uniform-cost search.

b. dfs is a special case of best-first tree search

true,
best-first tree search Expand a node with lowest h(n) first
when h(n) = - depth(n) , dfs is a special case of best-first tree search

c uniform –cost search is a special case of A * search

true,
A* expand a node with lowest f(n) = h（n）+g(n) first .
According to cost + h(n) expand.
uniform –cost search is a special case of A * search when h(n) == 0

problem3

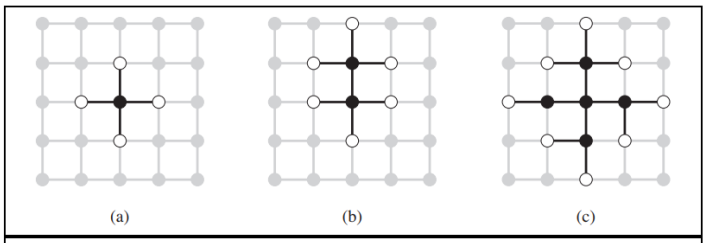


Figure 3.9 The separation property of GRAPH-SEARCH, illustrated on a rectangular-grid problem. The frontier (white nodes) always separates the explored region of the state space (black nodes) from the unexplored region (gray nodes). In (a), just the root has been expanded. In (b), one leaf node has been expanded. In (c), the remaining successors of the root have been expanded in clockwise order.

Consider the unbounded version of the regular 2D grid shown in Figure 3.9. The start state is at the origin, (0,0), and the goal state is at (x, y).

a. What is the branching factor b in this state space?

4

b. How many distinct states are there at depth k (for k > 0)?

At depth k, the number of distinct states is equal to the number of surface nodes. If we connect them, we will find that it is a Rhombus. The side length of the rhombus is k+1, and there are 4 repeated nodes in the corner.

In this way, we can figure out the number of distinct states = (K+1) * 4 -4 =4k

c. What is the maximum number of nodes expanded by breadth-first tree search?

breadth-first tree search doesn't have repeated state checking; it just expends four directions. For example, x=1, y =0. Nodes = 15,

$$\frac{1(1-4^2)}{1-4}$$

K+1 = 2 = |x| + |y| +1

So we can find the maximum number of nodes =

$$\frac{1(1-4^{|x|\ +\ |y|\ +1})}{1-4}$$

d. What is the maximum number of nodes expanded by breadth-first graph search?

breadth-first tree search has the repeated state checking.
X = 2, y = 2 ; the number of distinct states each depth= 4k
Sn = 1+ 4 first layer+ 8 second layer+12 +16 =(4+16)*4/2 = 40
Thus, the maximum number of nodes expanded by breadth-first graph search =

$$\frac{\left(4+4(|x|+|y|)\right)*(|x|+\ |y|)}{2}+1=\left(2+2(|x|+|y|)\right)*(|x|+\ |y|)$$

e. Is h = |u − x| + |v − y| an admissible heuristic for a state at (u, v)? Explain,

True, because this is actually Manhattan distance, smaller than other distance by searching.

f. How many nodes are expanded by A* graph search using h?

x+y

g. Does h remain admissible if some links are removed?

Yes, h（n）still smaller than real cost.

h. Does h remain admissible if some links are added between nonadjacent states?

False, these links will reduce the path distance, h(n) may bigger than the real cost.

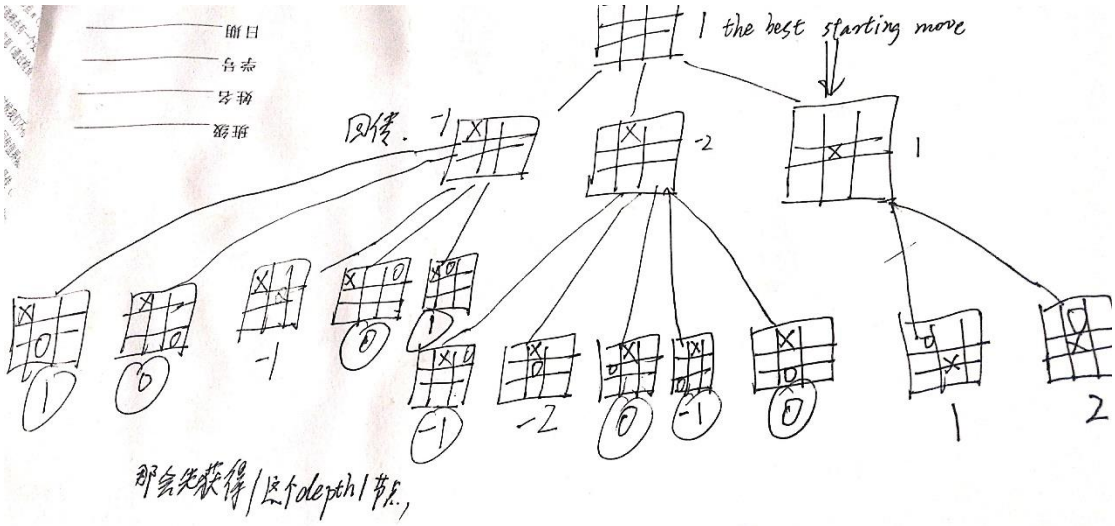
Problem 4

5.9 This problem exercises the basic concepts of game playing, using tic-tac-toe (noughts and crosses) as an example. We define Xn as the number of rows, columns, or diagonals with exactly n X's and no O's. Similarly, On is the number of rows, columns, or diagonals with just n O's. The utility function assigns +1 to any position with X3 = 1 and −1 to any position with O3 = 1. All other terminal positions have utility 0. For nonterminal positions, we use a linear

evaluation function defined as Eval(s)=3X2(s)+X1(s)−(3O2(s)+O1(s)).

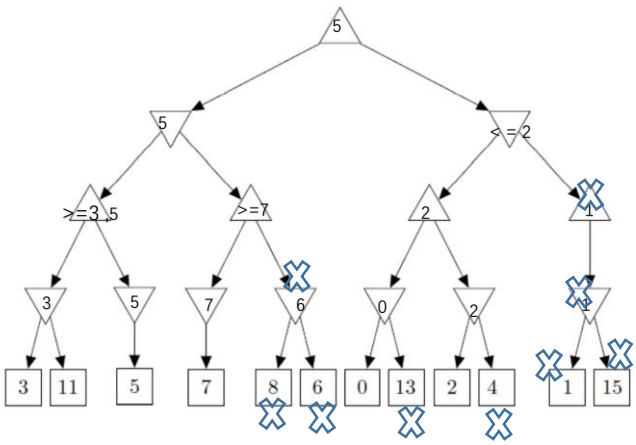
a. Approximately how many possible games of tic-tac-toe are there?
First step, have 9 choice, second step has 8 ···, the maximum number is 9 * 8 * 7 .. *1

b. Show the whole game tree starting from an empty board down to depth 2 (i.e., one X and one O on the board), taking symmetry into account.
c. Mark on your tree the evaluations of all the positions at depth 2.
d. Using the minimax algorithm, mark on your tree the backed-up values for the positions at depths 1 and 0, and use those values to choose the best starting move.
e. Circle the nodes at depth 2 that would not be evaluated if alpha–beta pruning was applied, assuming the nodes are generated in the optimal order for alpha–beta pruning.



Problem5

On the minimax game tree below cross out the branches removed by alpha-beta pruning assuming left to right traversal



We find left node 7, we needn't get 8,6 because right node has to>=7, min will choose 5.

We find 0, we needn't get 13 because min must < = 0 , while the root would not choose 0

because it has max 5 as an option. Similarly, we could prune 4.